

# LATENT DIMENSIONS

AP

# **DIMENSIONALITY REDUCTION**

## Problem 8: Data reduction

### Instance:

- a collection (dataset)  $\mathbf{D}$  of datapoints from  $\mathbf{X}$ , e.g.,  $\mathbb{R}^m$
- [a distinct independent variable  $x_i$ ]

**Solution:** a projection of  $\mathbf{D}$  onto  $\mathbb{R}^n, n < m$

**Measure:** error in the estimation of  $x_i$

Example: genre identification in consumer behaviour analysis

## EXAMPLE

Olivetti faces are 64x64 grayscale matrices.

Through SVD we discovered that most singular values are in fact 0 or very small

by considering only the top 20 or so singular values we can obtain a very similar image with less data:

- $A$  contains  $64 \times 64 = 4096$  values
- $U\Sigma V^T$  contains  $20 \times 20 + 20 + 20 \times 20 = 820$  values (5:1 compression ratio)

# WHY REDUCE DIMENSIONS?

- easier to store, quicker to process
- interpretation and visualisation
- remove redundant or noisy features
- escape the curse of dimensionality and go back to intuitive distance features
- discover hidden correlations/topics

## WHEN REDUCE DIMENSIONS?

When any of the goals becomes important

Empirically: when we believe that data essentially represents the mixing of a smaller set of *feature* which are the real *axes* of the data.

There really are only two *features*: sci-fi and romance

|       | Matrix | Alien | Star Wars | Casablanca | Titanic |
|-------|--------|-------|-----------|------------|---------|
| Joe   | 1      | 1     | 1         | 0          | 0       |
| Jim   | 3      | 3     | 3         | 0          | 0       |
| John  | 4      | 4     | 4         | 0          | 0       |
| Jack  | 5      | 5     | 5         | 0          | 0       |
| Jill  | 0      | 0     | 0         | 4          | 4       |
| Jenny | 0      | 0     | 0         | 5          | 5       |
| Jane  | 0      | 0     | 0         | 2          | 2       |

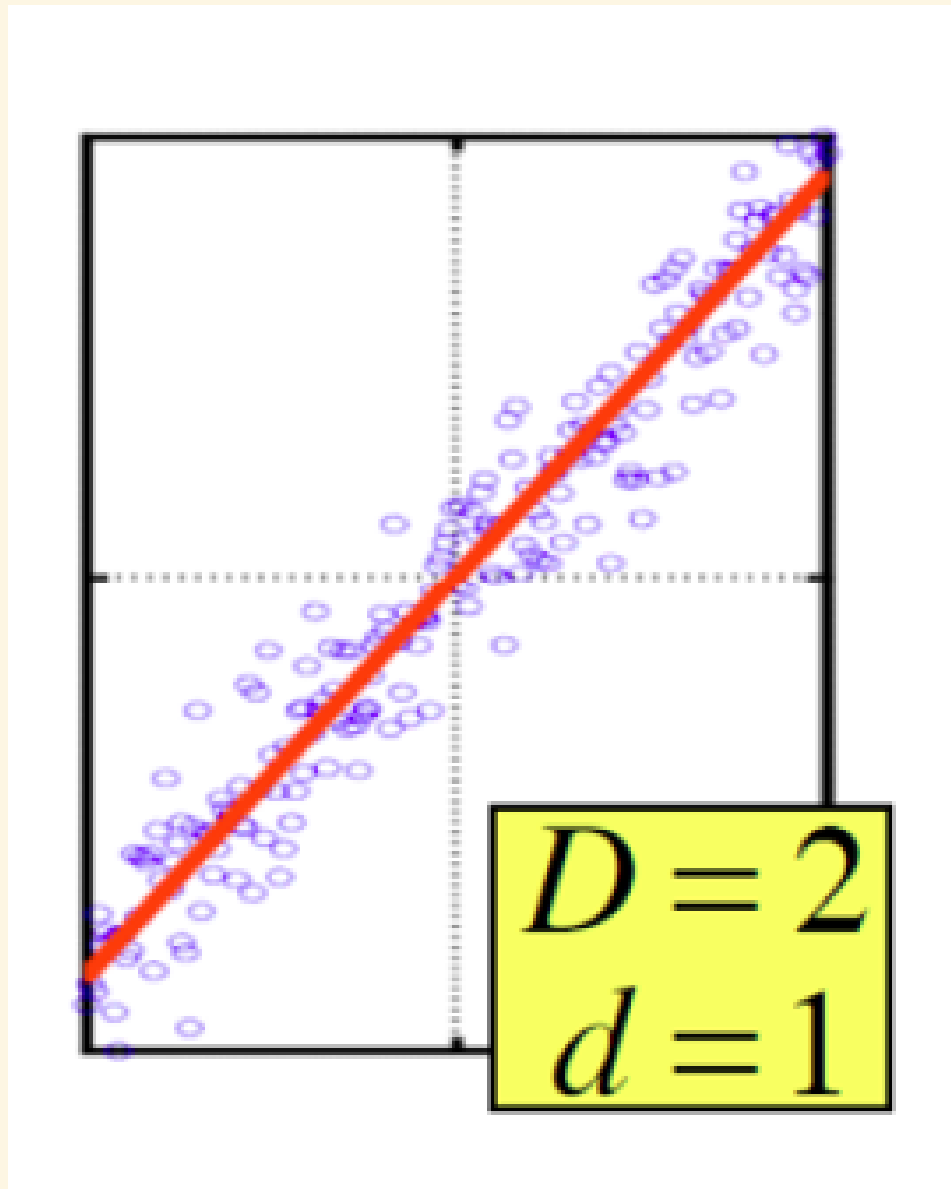
Figure 11.6: Ratings of movies by users

# HOW TO REDUCE DIMENSIONS?

High-level view:

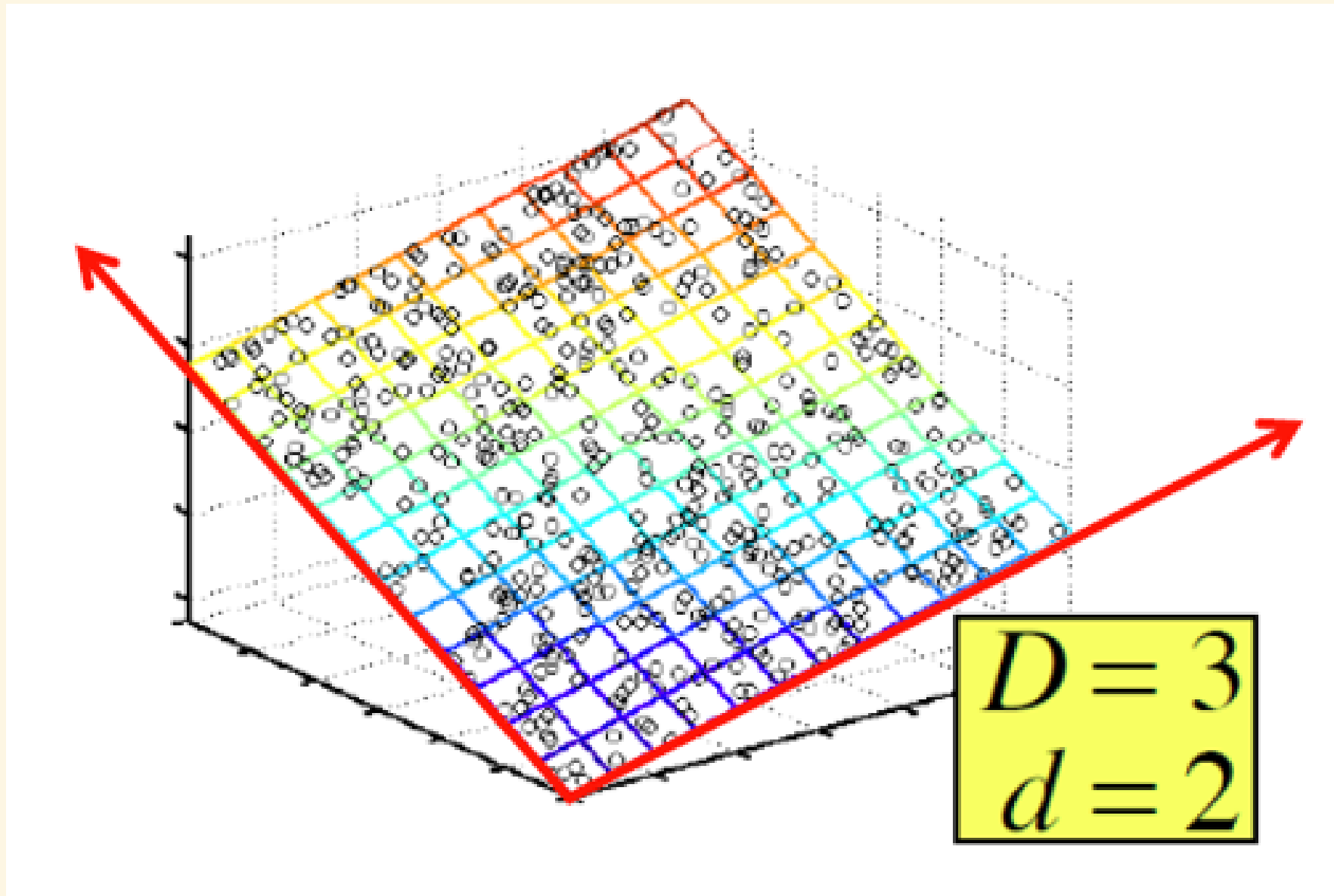
- data lies on or near a low-dimensional space:
- axes of that space are effective representations of data

EXAMPLES:  $D \longrightarrow d$





The data axes (in red) are almost never the original measurement axes



## BACKGROUND: MATRIX RANK

Rank: the max number of columns (or rows) that are linearly independent.

Independent cols/rows are *candidates for the new, reduced reference system* (the red axes)

For rectangular matrices, SVD uses  $r \leq \min\{n, m\}$

We can map data points into a completely-new, dataset-dependent representation!

The dataset-dependent representation will be compact

Let's create a new, abstract *feature space*

## EXAMPLE

Handmade dimensionality reduction, from the MMDS textbook

$$A_{3 \times 3} = \begin{pmatrix} 1 & 2 & 1 \\ -2 & -3 & 1 \\ 3 & 5 & 0 \end{pmatrix}$$

$r = 2$  as the third row can be expressed as the first minus the second:

$$\begin{array}{rrrr} 1 & 2 & 1 & - \\ -2 & -3 & 1 & = \\ \hline 3 & 5 & 0 & \end{array}$$

We create a new 2-d space where the axes are the first two rows:

$$[1, 0, 0], [0, 1, 0], [0, 0, 1] \longrightarrow [1, 2, 1], [-2, -3, 1]$$

The new rows are  $[1, 0]$   $[0, 1]$  and  $[1, 1]$

$$A'_{3 \times 2} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & -1 \end{pmatrix}$$

The new points work as selectors of the new axes: it's easy to go back from this space to the original, no loss of information/precision

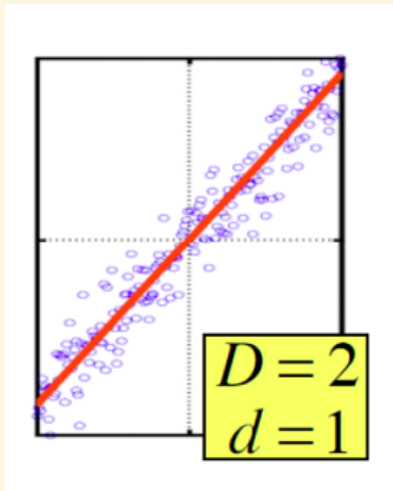
# DIMENSIONALITY REDUCTION

In real dataset mapping to a lower-dimensional space may introduce errors in the  $2 \rightarrow 1$  example below, instead of points we just take the measure (distance from the origin) of their projection on the red axis

The red axis is chosen to minimise the error introduced by the  $2 \rightarrow 1$  projection.

Data mining studies how to find such axes, called *concepts*

They capture some alignment which is inherent to the data.



# SVD

# DECOMPOSITION

$$A_{(m \times n)} = U_{(m \times m)} D_{(m \times n)} V_{(n \times n)}^T$$

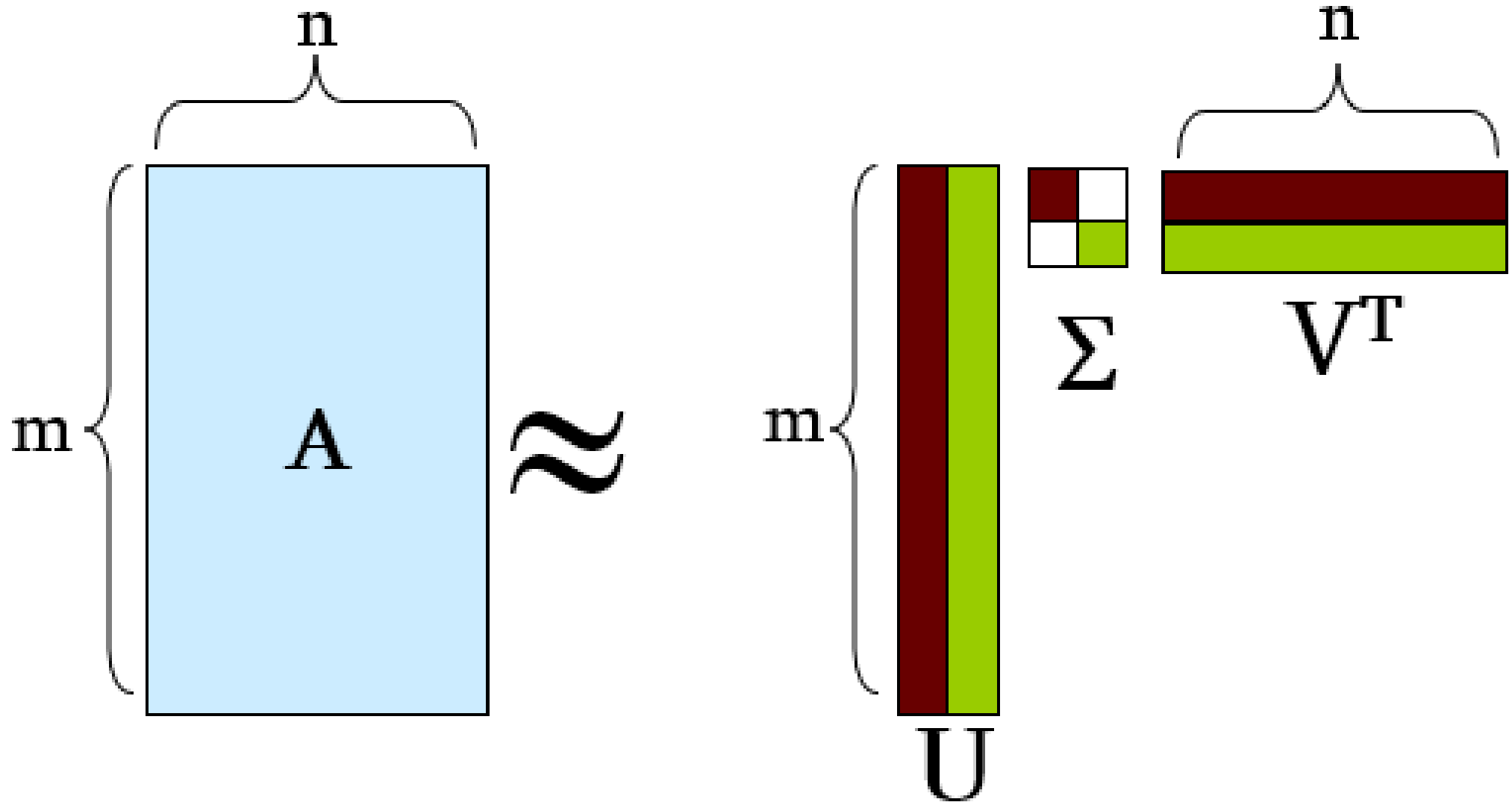
- U is a orthogonal m. of *left-singular* (col.) vectors
- D is a diagonal matrix of *singular values*
- V is a orthogonal m. of *right-singular* (col.) vectors

Suppose only  $r$  ( $r < \min\{m, n\}$ ) singular values are non-zero

We can rewrite the decomposition as follows:

$$A_{(m \times n)} \approx U_{(m \times r)} D_{(r \times r)} V_{(r \times n)}^T$$

Suppose  $r = 2$ , visualise  $A_{(m \times n)} \approx U_{(m \times 2)} D_{(2 \times 2)} V_{(2 \times n)}^T$

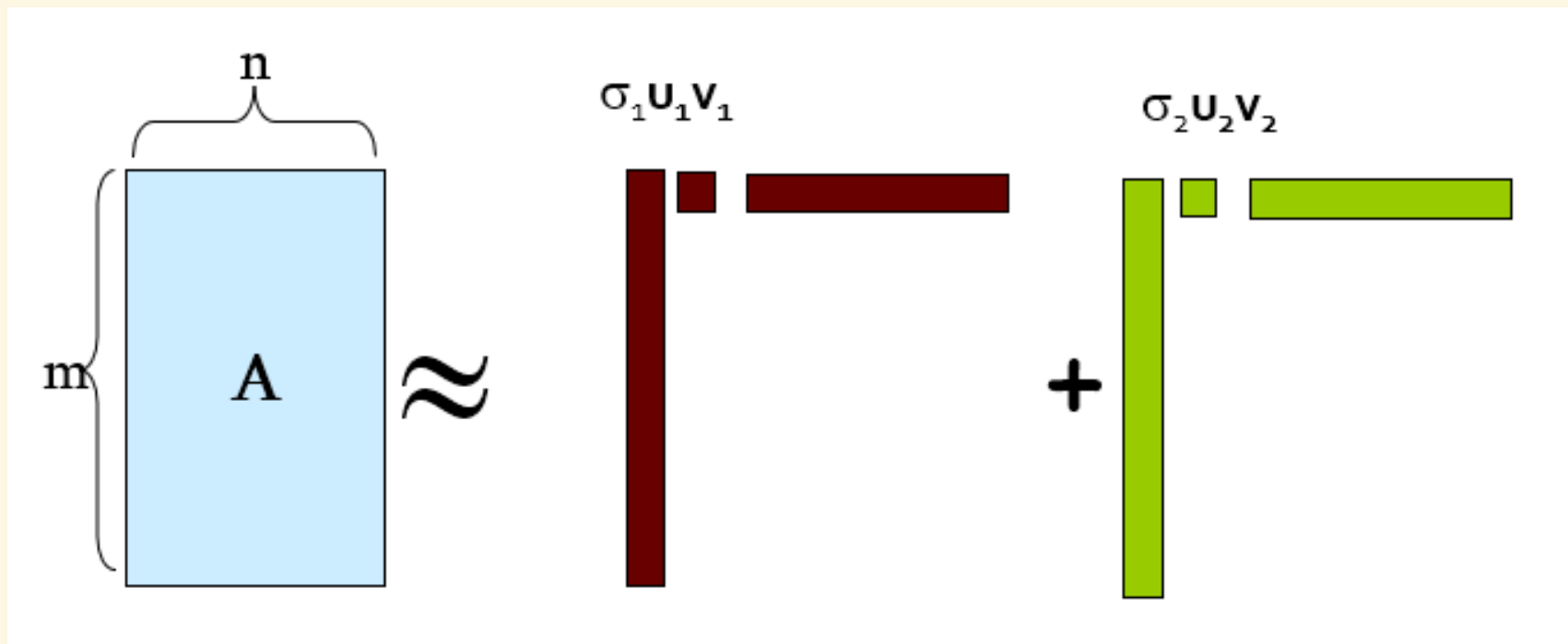




$$A_{(m \times n)} \approx \sum_i \sigma_i U_i \circ V_i^T$$

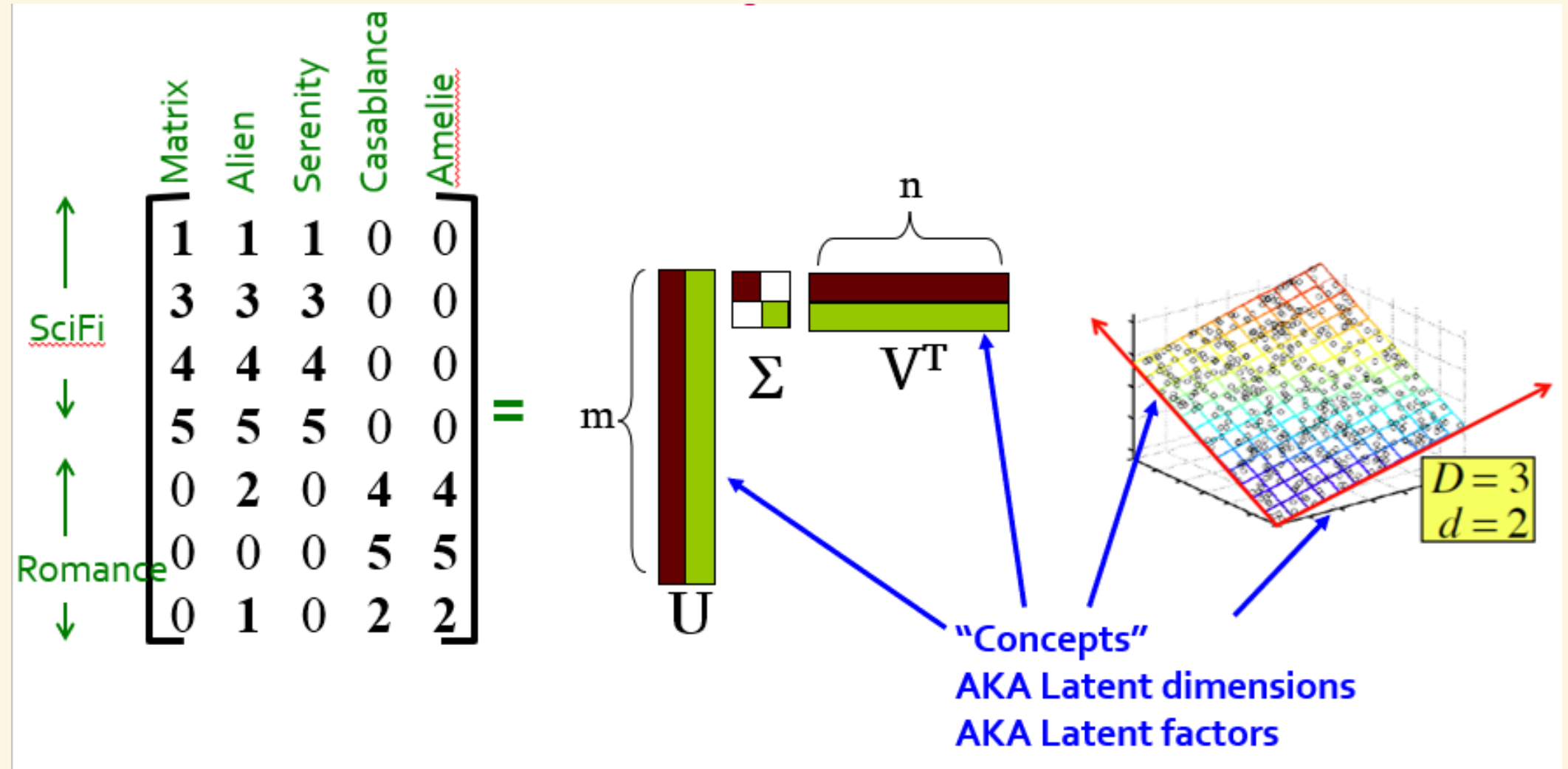
where  $U_i$  is the  $i$ -th column of  $U$  and  $\circ$  is matrix multiplication

Now  $A$  is represented as the sum of independent *factors* that were not explicit in the original data



# USERS-TO-FILMS BECOMES USERS-TO-CONCEPTS-TO-FILMS

(slightly different data)

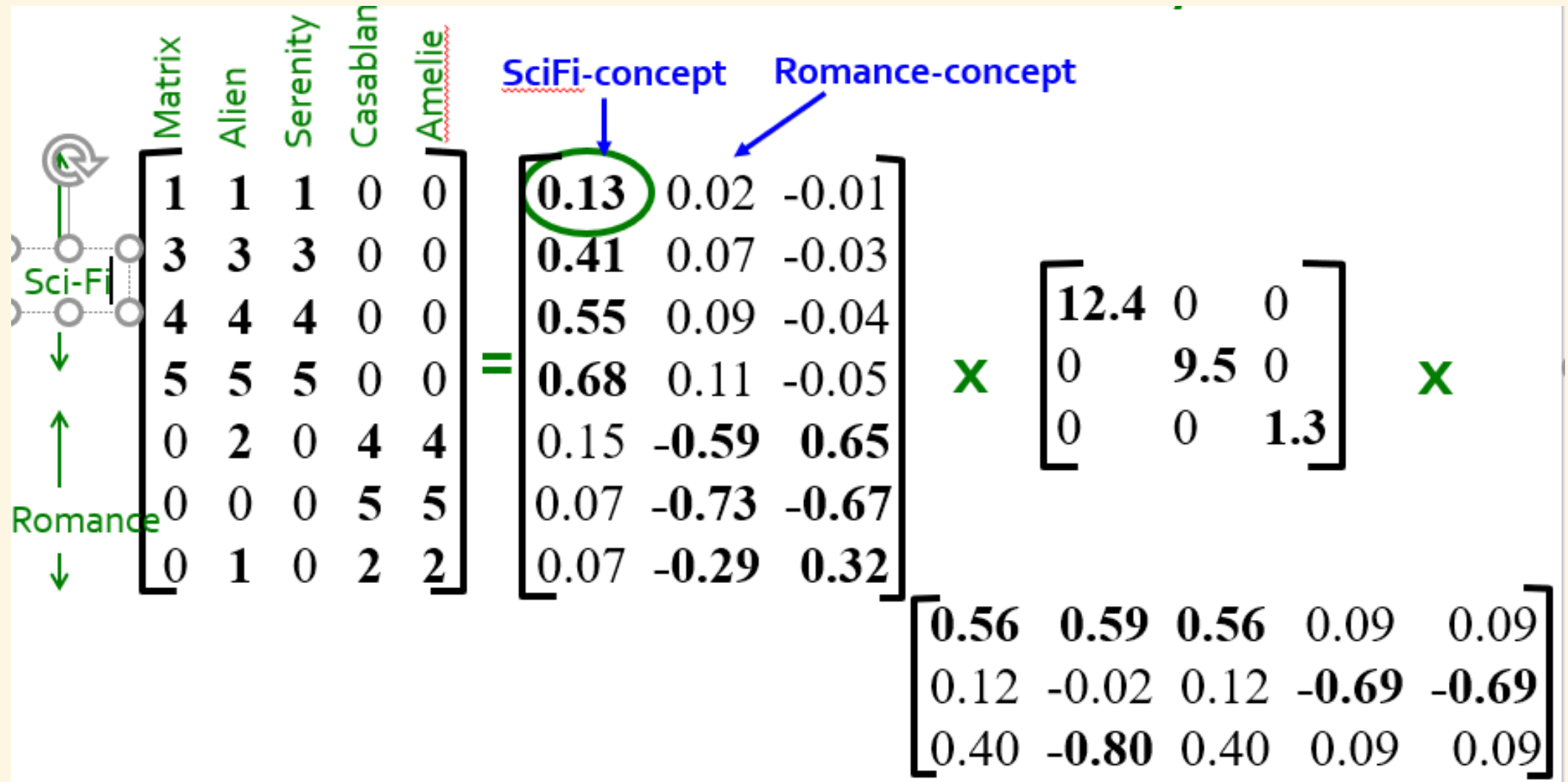


$$\begin{array}{c}
 \uparrow \\
 \text{SciFi} \\
 \downarrow \\
 \uparrow \\
 \text{Romance} \\
 \downarrow
 \end{array}
 \begin{array}{c}
 \text{Matrix} \\
 \text{Alien} \\
 \text{Serenity} \\
 \text{Casablanca} \\
 \text{Amelie}
 \end{array}
 \begin{bmatrix}
 1 & 1 & 1 & 0 & 0 \\
 3 & 3 & 3 & 0 & 0 \\
 4 & 4 & 4 & 0 & 0 \\
 5 & 5 & 5 & 0 & 0 \\
 0 & 2 & 0 & 4 & 4 \\
 0 & 0 & 0 & 5 & 5 \\
 0 & 1 & 0 & 2 & 2
 \end{bmatrix}
 =
 \begin{bmatrix}
 0.13 & 0.02 & -0.01 \\
 0.41 & 0.07 & -0.03 \\
 0.55 & 0.09 & -0.04 \\
 0.68 & 0.11 & -0.05 \\
 0.15 & -0.59 & 0.65 \\
 0.07 & -0.73 & -0.67 \\
 0.07 & -0.29 & 0.32
 \end{bmatrix}
 \times
 \begin{bmatrix}
 12.4 & 0 & 0 \\
 0 & 9.5 & 0 \\
 0 & 0 & 1.3
 \end{bmatrix}
 \times
 \begin{bmatrix}
 0.56 & 0.59 & 0.56 & 0.09 & 0.09 \\
 0.12 & -0.02 & 0.12 & -0.69 & -0.69 \\
 0.40 & -0.80 & 0.40 & 0.09 & 0.09
 \end{bmatrix}$$

# CONCEPTS

$$\begin{array}{c}
 \uparrow \\
 \text{Sci-Fi} \\
 \downarrow \\
 \uparrow \\
 \text{Romance} \\
 \downarrow
 \end{array}
 \begin{array}{c}
 \text{Matrix} \\
 \text{Alien} \\
 \text{Serenity} \\
 \text{Casablanca} \\
 \text{Amelie}
 \end{array}
 \begin{bmatrix}
 1 & 1 & 1 & 0 & 0 \\
 3 & 3 & 3 & 0 & 0 \\
 4 & 4 & 4 & 0 & 0 \\
 5 & 5 & 5 & 0 & 0 \\
 0 & 2 & 0 & 4 & 4 \\
 0 & 0 & 0 & 5 & 5 \\
 0 & 1 & 0 & 2 & 2
 \end{bmatrix}
 =
 \begin{array}{c}
 \text{SciFi-concept} \\
 \text{Romance-concept}
 \end{array}
 \begin{bmatrix}
 0.13 & 0.02 & -0.01 \\
 0.41 & 0.07 & -0.03 \\
 0.55 & 0.09 & -0.04 \\
 0.68 & 0.11 & -0.05 \\
 0.15 & -0.59 & 0.65 \\
 0.07 & -0.73 & -0.67 \\
 0.07 & -0.29 & 0.32
 \end{bmatrix}
 \times
 \begin{bmatrix}
 12.4 & 0 & 0 \\
 0 & 9.5 & 0 \\
 0 & 0 & 1.3
 \end{bmatrix}
 \times
 \begin{bmatrix}
 0.56 & 0.59 & 0.56 & 0.09 & 0.09 \\
 0.12 & -0.02 & 0.12 & -0.69 & -0.69 \\
 0.40 & -0.80 & 0.40 & 0.09 & 0.09
 \end{bmatrix}$$

$U$  is a user-to-concept similarity matrix



Matrix

|   | Matrix | Alien | Serenity | Casablan | Amelie |
|---|--------|-------|----------|----------|--------|
| 1 | 1      | 1     | 1        | 0        | 0      |
| 3 | 3      | 3     | 3        | 0        | 0      |
| 4 | 4      | 4     | 4        | 0        | 0      |
| 5 | 5      | 5     | 5        | 0        | 0      |
| 0 | 2      | 0     | 4        | 4        | 4      |
| 0 | 0      | 0     | 5        | 5        | 5      |
| 0 | 1      | 0     | 2        | 2        | 2      |

Sci-Fi-concept

Romance-concept

Sci-Fi

Romance

0.13

0.02

-0.01

0.41

0.07

-0.03

0.55

0.09

-0.04

0.68

0.11

-0.05

0.15

-0.59

0.65

0.07

-0.73

-0.67

0.07

-0.29

0.32

12.4

0

0

0

9.5

0

0

0

1.3

0.56

0.59

0.56

0.09

0.09

0.12

-0.02

0.12

-0.69

-0.69

0.40

-0.80

0.40

0.09

0.09

$\sigma$ s reveal the strength of each concept

$$\begin{array}{c}
 \begin{array}{c} \uparrow \\ \text{SciFi} \\ \downarrow \\ \uparrow \\ \text{Romnce} \\ \downarrow \end{array}
 \begin{bmatrix}
 \text{Matrix} & \text{Alien} & \text{Serenity} & \text{Casablanca} & \text{Amelie} \\
 1 & 1 & 1 & 0 & 0 \\
 3 & 3 & 3 & 0 & 0 \\
 4 & 4 & 4 & 0 & 0 \\
 5 & 5 & 5 & 0 & 0 \\
 0 & 2 & 0 & 4 & 4 \\
 0 & 0 & 0 & 5 & 5 \\
 0 & 1 & 0 & 2 & 2
 \end{bmatrix}
 =
 \begin{array}{c}
 \text{SciFi-concept} \\
 \downarrow \\
 \begin{bmatrix}
 0.13 & 0.02 & -0.01 \\
 0.41 & 0.07 & -0.03 \\
 0.55 & 0.09 & -0.04 \\
 0.68 & 0.11 & -0.05 \\
 0.15 & -0.59 & 0.65 \\
 0.07 & -0.73 & -0.67 \\
 0.07 & -0.29 & 0.32
 \end{bmatrix}
 \end{array}
 \times
 \begin{array}{c}
 \text{"strength" of the SciFi-concept} \\
 \downarrow \\
 \begin{bmatrix}
 12.4 & 0 & 0 \\
 0 & 9.5 & 0 \\
 0 & 0 & 1.3
 \end{bmatrix}
 \end{array}
 \times
 \begin{bmatrix}
 0.56 & 0.59 & 0.56 & 0.09 & 0.09 \\
 0.12 & -0.02 & 0.12 & -0.69 & -0.69 \\
 0.40 & -0.80 & 0.40 & 0.09 & 0.09
 \end{bmatrix}
 \end{array}$$

$V^T$  is a concept-to-film similarity matrix

The diagram illustrates the calculation of a film's concept vector using matrix multiplication. It shows the relationship between Sci-Fi and Romance concepts and how they relate to specific films.

**Sci-Fi to Film Similarity Matrix ( $V^T$ ):**

|         | Matrix | Alien | Serenity | Casablanca | Amelie |
|---------|--------|-------|----------|------------|--------|
| Sci-Fi  | 1      | 1     | 1        | 0          | 0      |
|         | 3      | 3     | 3        | 0          | 0      |
|         | 4      | 4     | 4        | 0          | 0      |
|         | 5      | 5     | 5        | 0          | 0      |
| Romance | 0      | 2     | 0        | 4          | 4      |
|         | 0      | 0     | 0        | 5          | 5      |
|         | 0      | 1     | 0        | 2          | 2      |

**Sci-Fi Concept Vector:**

$$\begin{bmatrix} 0.13 & 0.02 & -0.01 \\ 0.41 & 0.07 & -0.03 \\ 0.55 & 0.09 & -0.04 \\ 0.68 & 0.11 & -0.05 \\ 0.15 & -0.59 & 0.65 \\ 0.07 & -0.73 & -0.67 \\ 0.07 & -0.29 & 0.32 \end{bmatrix}$$

**Sci-Fi to Romance Similarity Matrix:**

$$\begin{bmatrix} 12.4 & 0 & 0 \\ 0 & 9.5 & 0 \\ 0 & 0 & 1.3 \end{bmatrix}$$

**Romance Concept Vector:**

$$\begin{bmatrix} 0.56 & 0.59 & 0.56 & 0.09 & 0.09 \\ 0.12 & -0.02 & 0.12 & -0.69 & -0.69 \\ 0.40 & -0.80 & 0.40 & 0.09 & 0.09 \end{bmatrix}$$

The calculation is shown as:

$$\begin{bmatrix} 0.13 & 0.02 & -0.01 \\ 0.41 & 0.07 & -0.03 \\ 0.55 & 0.09 & -0.04 \\ 0.68 & 0.11 & -0.05 \\ 0.15 & -0.59 & 0.65 \\ 0.07 & -0.73 & -0.67 \\ 0.07 & -0.29 & 0.32 \end{bmatrix} \times \begin{bmatrix} 12.4 & 0 & 0 \\ 0 & 9.5 & 0 \\ 0 & 0 & 1.3 \end{bmatrix} = \begin{bmatrix} 0.56 & 0.59 & 0.56 & 0.09 & 0.09 \\ 0.12 & -0.02 & 0.12 & -0.69 & -0.69 \\ 0.40 & -0.80 & 0.40 & 0.09 & 0.09 \end{bmatrix}$$

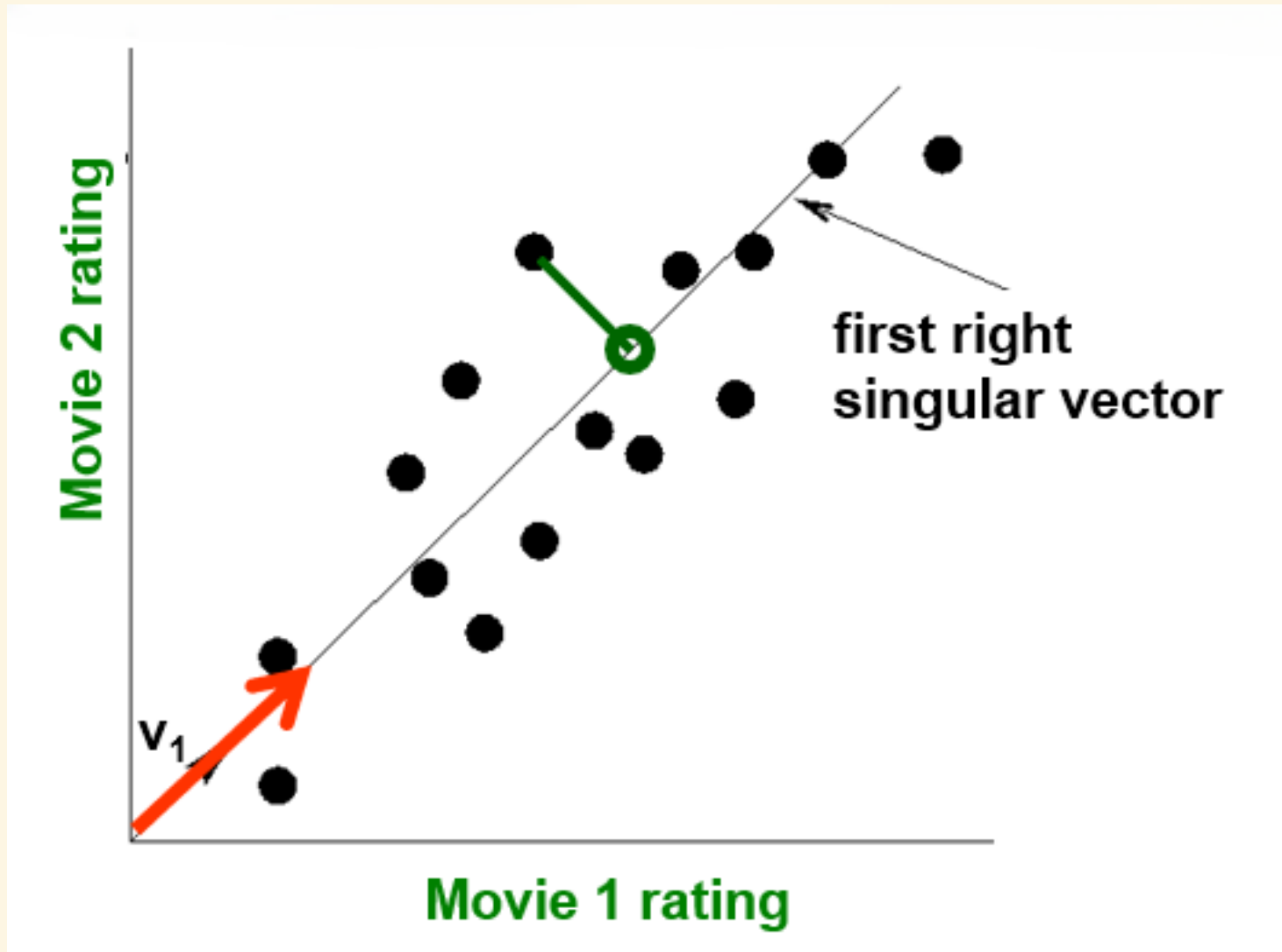
Arrows indicate the flow of information: the Sci-Fi concept vector is multiplied by the Sci-Fi to Romance similarity matrix to produce the Romance concept vector. The Sci-Fi concept vector is also derived from the Sci-Fi to Film similarity matrix and the Film to Romance similarity matrix.

# SVD INTERPRETATION -1



# FIRST INTERPRETATION

The *singular vectors* that make up  $V$  (and  $U$ ) are the new axes for projection

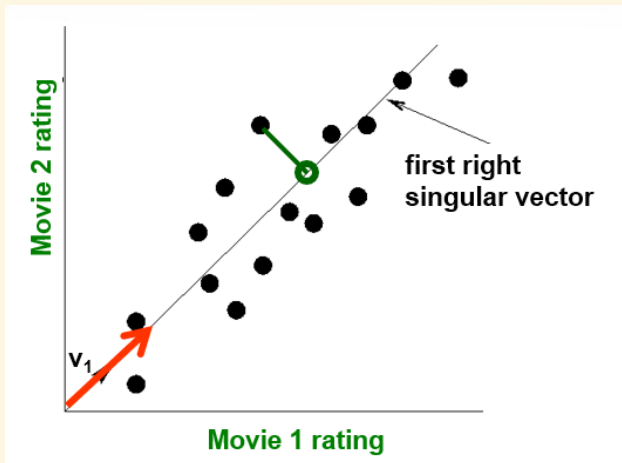


The *singular vectors* that make up  $V$  (and  $U$ ) are the new axes for projection

They will minimise the *reconstruction error* ( $z$  is the value obtained by the reduced SVD)

$$\epsilon = \sum_{i=1}^m \sum_{j=1}^n \|a_{ij} - z_{ij}\|$$

The sum of the green, orthogonal lines connecting the point to its compressed representation on the new axis is minimal

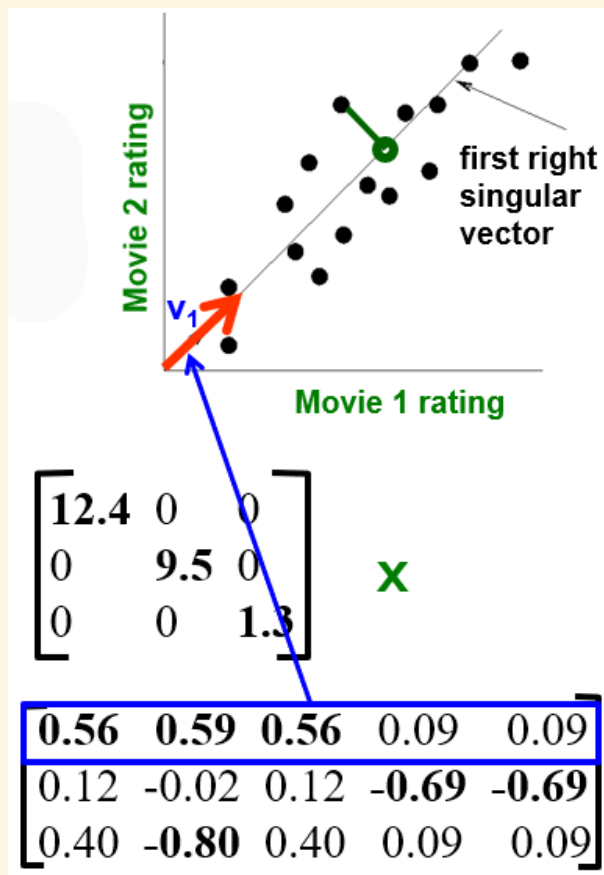


# SVD INTERPRETATION -2

## SECOND INTERPRETATION

Again, we use the singular vectors as the new axes

The  $\sigma$ s indicate the *spread* (variance) of the projected points on the new axis  
the more spread apart points are, the easier it will be to classify/find cluster them



Let's see what happens to the original datapoints

$UD$  project users onto the *concept* axes

$$\begin{bmatrix} \mathbf{0.13} & 0.02 & -0.01 \\ \mathbf{0.41} & 0.07 & -0.03 \\ \mathbf{0.55} & 0.09 & -0.04 \\ \mathbf{0.68} & 0.11 & -0.05 \\ 0.15 & \mathbf{-0.59} & \mathbf{0.65} \\ 0.07 & \mathbf{-0.73} & \mathbf{-0.67} \\ 0.07 & \mathbf{-0.29} & \mathbf{0.32} \end{bmatrix} \times \begin{bmatrix} \mathbf{12.4} & 0 & 0 \\ 0 & \mathbf{9.5} & 0 \\ 0 & 0 & \mathbf{1.3} \end{bmatrix}$$

We see values that *look like* measures of the implicit user-to-topic affiliation

Interpreting negative values is problematic, especially for  $V^T$

$$\begin{bmatrix} 0.13 & 0.02 & -0.01 \\ 0.41 & 0.07 & -0.03 \\ 0.55 & 0.09 & -0.04 \\ 0.68 & 0.11 & -0.05 \\ 0.15 & -0.59 & 0.65 \\ 0.07 & -0.73 & -0.67 \\ 0.07 & -0.29 & 0.32 \end{bmatrix} \times \begin{bmatrix} 12.4 & 0 & 0 \\ 0 & 9.5 & 0 \\ 0 & 0 & 1.3 \end{bmatrix}$$

Projection of users  
on the “Sci-Fi” axis  
( $U \Sigma^T$ ):

|      | Movie 1 rating |       |  |
|------|----------------|-------|--|
| 1.61 | 0.19           | -0.01 |  |
| 5.08 | 0.66           | -0.03 |  |
| 6.82 | 0.85           | -0.05 |  |
| 8.43 | 1.04           | -0.06 |  |
| 1.86 | -5.60          | 0.84  |  |
| 0.86 | -6.93          | -0.87 |  |
| 0.86 | -2.75          | 0.41  |  |

# **DIMENSIONALITY REDUCTION**

## SET THE SMALLEST SINGULAR VALUES TO 0

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 3 & 3 & 3 & 0 & 0 \\ 4 & 4 & 4 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 \\ 0 & 2 & 0 & 4 & 4 \\ 0 & 0 & 0 & 5 & 5 \\ 0 & 1 & 0 & 2 & 2 \end{bmatrix} \approx \begin{bmatrix} 0.13 & 0.02 & -0.01 \\ 0.41 & 0.07 & -0.03 \\ 0.55 & 0.09 & -0.04 \\ 0.68 & 0.11 & -0.05 \\ 0.15 & -0.59 & 0.65 \\ 0.07 & -0.73 & -0.67 \\ 0.07 & -0.29 & 0.32 \end{bmatrix} \times \begin{bmatrix} 12.4 & 0 & 0 \\ 0 & 9.5 & 0 \\ 0 & 0 & 1.3 \end{bmatrix} \times \begin{bmatrix} 0.56 & 0.59 & 0.56 & 0.09 & 0.09 \\ 0.12 & -0.02 & 0.12 & -0.69 & -0.69 \\ 0.40 & -0.80 & 0.40 & 0.09 & 0.09 \end{bmatrix}$$

The image illustrates the process of setting the smallest singular values to zero in a matrix decomposition. The first matrix is a 7x5 matrix. The second matrix is a 7x3 matrix of singular values, with the third column (containing the smallest singular values) crossed out with a red 'X'. The third matrix is a 3x3 matrix of singular vectors, with the third row (corresponding to the zeroed-out singular value) crossed out with a red 'X'. The final matrix is a 7x5 matrix of reconstructed components, with the third row (corresponding to the zeroed-out singular value) crossed out with a red 'X'.



# THE B MATRIX

Let  $A = UDV^T$  as before, with a ranking  $r \leq \min\{m, n\}$

For a small integer  $k$ , define

and let  $E$  be a *reduction* of  $D$ : same everywhere but for  $\sigma_{k+1} \dots \sigma_r$  set to 0.

Theorem:  $B = UEV^T$  with ranking  $k$  is the best approximation of  $A$  with rank  $k$ :

$$B \in \operatorname{argmin}_{X: \operatorname{rank}(X)=k} \{ \|A - X\|_F \}$$

$$\begin{pmatrix} x_{11} & x_{12} & \dots & x_{1n} \\ x_{21} & x_{22} & \dots & \\ \vdots & \vdots & \ddots & \\ x_{m1} & & & x_{mn} \end{pmatrix}_{m \times n} = \begin{pmatrix} u_{11} & \dots & u_{1r} \\ \vdots & \ddots & \vdots \\ u_{m1} & & u_{mr} \end{pmatrix}_{m \times r} \begin{pmatrix} \sigma_{11} & 0 & \dots \\ 0 & \ddots & \\ \vdots & & \end{pmatrix}_{r \times r} \begin{pmatrix} v_{11} & \dots & v_{1n} \\ \vdots & \ddots & \\ & & \end{pmatrix}_{r \times n}$$

## WHY B IS A GOOD REDUCTION?

rows of  $U$  and  $V$  are the new axes, but they are unit vectors

the singular values  $\sigma_i$  do the scaling

$$A_{(m \times n)} \approx \sigma_1 U_1 \circ V_1^T + \sigma_2 U_2 \circ V_2^T + \dots \sigma_r U_r \circ V_r^T$$

since we constructed  $D$  to have  $\sigma_1 \geq \sigma_2 \dots$ , the smallest  $\sigma$  will do little scaling

so dropping  $\sigma_{k+1} \dots \sigma_r$  will introduce less error

Rule of thumb: keep singular values that sum up to about 80% of the total 'energy'

$$\sum_{i=1}^k \sigma_i \approx 0.8 \sum_{j=1}^r \sigma_j$$

# CONCLUSIONS

$A = UDV^T$  provides a unique decomposition that is *interpretable*

**Interpretability:**

$U$ : user-to-concept affiliations/similarities

$D$ : strength of each concept

$V$ : film-to-concept affiliations/similarities

SVD picks up linear dependencies

## Dimensionality reduction:

SVD finds the *best* reduced matrix  $B$

$k$  is not really an hyperparameter (as with K-nn): we set it empirically to keep 80% of values

SVD dim. reduction is the key to *denoising*

## Cost:

the core component requires  $\min\{nm^2, n^2m\}$  ops.

## Implementation:

truncatedSVD in Scikit-learn

```
1 from sklearn.decomposition import TruncatedSVD
2
3 K=2
4
5 % X is assigned ...
6
7 svd = TruncatedSVD(n_components=K)
8
9 X_reduced = svd.fit_transform(X)
```